Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Design Introduction

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

INDEX

1.1.1 Java Introduction

- Java is a programming language and a platform. Java is a high level, robust, object-oriented and secure programming language.
- Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995. James Gosling is known as the father of Java. Before Java, its name was Oak. Since Oak was already a registered company, so James Gosling and his team changed the name from Oak to Java.
- Platform: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

- Let's have a quick look at Java programming example. A detailed description of Hello Java example is available in next page.
- Simple.java

```
class Simple
{
  public static void main(String args[])
  {
        System.out.println("Hello Java");
  }
}
```

1.1.2. Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects rather than functions and logic. An object is a combination of data (attributes) and methods (functions) that operate on that data.

🔑 **Key Concepts of OOP:**

1. **Class: A blueprint for creating objects. It defines properties (variables) and behaviors (methods).**
2. **Object: An instance of a class. It holds actual values and can perform actions.**
3. **Encapsulation: Binding of data and methods together and restricting direct access to some components.**
4. **Inheritance: Mechanism where one class acquires the properties and behaviors of another.**
5. **Polymorphism: Ability of objects to take on many forms, typically through method overloading and overriding.**
6. **Abstraction: Hiding internal details and showing only essential features to the user.**

📦 **Real-Life Example:**

Think of a Car as an object:

- **Data (Attributes): color, model, speed**
- **Methods (Behaviors): start(), stop(), accelerate()**

Simple Java Example:

```
class Car {
   String color;
   void start() {
      System.out.println("Car started");
   }
}
public class Main {
   public static void main(String[] args) {
      Car myCar = new Car();
      myCar.color = "Red";
      myCar.start();
   }
}
```

1.1.3. OOP Principles

OOP (Object-Oriented Programming) principles are the fundamental concepts that define how objects interact and how software is designed in object-oriented languages like Java. These principles help make code more modular, reusable, and easier to maintain.

🔑 Four Main Principles of OOP:

1. Encapsulation

Definition: Wrapping data (variables) and methods (functions) into a single unit called a class, and restricting access to some components.

- **Goal: Protect internal object state and prevent unauthorized access.**
- **How: Use access modifiers (private, public) and getter/setter methods.**

Example:

```
class Person {
    private String name;
    public void setName(String n) { name = n; }
    public String getName() { return name; }
 }
```

2. Inheritance

Definition: Mechanism where one class (child/subclass) inherits properties and methods from another class (parent/superclass).

- **Goal**: Promote code reuse and hierarchical classification.
- **How**: Use the extends keyword in Java.

Example:

```
class Animal {
   void eat() { System.out.println("This animal eats food."); }
}

class Dog extends Animal {
   void bark() { System.out.println("Dog barks."); }
}
```

3. Polymorphism

Definition: The ability of an object to take on many forms; same method behaves differently on different classes.

- **Types:**
  - **Compile-time (Method Overloading)**
  - **Run-time (Method Overriding)**

Example (Overriding):

```
class Animal {
   void sound() { System.out.println("Animal makes sound"); }
}
class Cat extends Animal {
   void sound() { System.out.println("Meow"); }
}
```

4. Abstraction

Definition: Hiding the complex implementation details and showing only essential features of the object.

- **Goal: Focus on *what* an object does, not *how* it does it.**
- **How: Using abstract classes or interfaces.**

Example:

```
abstract class Shape {
   abstract void draw();
}

class Circle extends Shape {
   void draw() { System.out.println("Drawing circle"); }
}
```

📊 Summary Table:

Principle	Purpose	Example Keyword
Encapsulation	Protect data, control access	private, getters/setters
Inheritance	Reuse code, establish relationships	extends
Polymorphism	Use same interface in many forms	Overloading, Overriding
Abstraction	Hide details, show only essentials	abstract, interface

1.2.1. Java as an Object-Oriented Programming (OOP) Language

Java is fundamentally built on the Object-Oriented Programming paradigm, which promotes modular and scalable software design.

Key Features:

- **Everything in Java (except primitives) is treated as an object.**
- **Java supports all major OOP principles:**
  - **Encapsulation – Classes and access modifiers.**
  - **Inheritance – Using extends for class hierarchy.**
  - **Polymorphism – Overloading and overriding.**
  - **Abstraction – Abstract classes and interfaces.**

🔶 Java's OOP structure encourages **code reuse**, **maintainability**, and **readability**.

1.2.2. 🌐 Java as an Internet-Enabled Language

Java was designed with **networking and web support** in mind, making it a powerful tool for internet-based applications.

Key Features:

- **Networking APIs: java.net package supports TCP/IP, UDP, sockets, etc.**
- **Applets (Legacy): Java programs that could run in browsers. Platform Independence: Java programs run on any system with the JVM ("Write Once, Run Anywhere").**
- **Security: Java provides a security manager and bytecode verification.**
- **Support for distributed computing: Technologies like RMI (Remote Method Invocation), CORBA.**

Real-life Usage:

- **Web services**
- **Web applications with Java Servlets, JSP**
- **Cloud and enterprise systems using Spring, Jakarta EE**

1.2.3 Importance of Java in Modern Software Development

Java remains one of the most widely used languages across multiple domains.

Key Reasons:

- **Platform Independence: JVM allows execution on any OS.**
- **Large Ecosystem: Rich libraries, tools (Maven, Gradle), and frameworks (Spring, Hibernate).**
- **Enterprise-Ready: Reliable for banking, e-commerce, telecom, ERP systems.**
- **Android Development: Java is a primary language for Android apps.**
- **Community Support: Massive global developer community and resources.**

Domains where Java excels:

- **Web Development (e.g., Spring, JSP)**
- **Mobile Apps (Android)**
- **Big Data (e.g., Hadoop is Java-based)**
- **Cloud Computing**
- **Embedded Systems**

3. Summary of Key Concepts:

- **OOP is a paradigm based on objects that combine data and behavior.**
- **Key Principles:**
  - **Encapsulation – Hides internal details using classes and access control.**
  - **Inheritance – Enables reuse by deriving new classes from existing ones.**
  - **Polymorphism – Allows the same method to behave differently in different contexts.**
  - **Abstraction – Focuses on essential features while hiding complexity.**

Java as an OOP Language: Fully supports OOP principles for building modular, reusable applications.

Java as an Internet-Enabled Language: Provides built-in networking, platform independence (via JVM), and security features.

Importance of Java: Widely used in web, mobile (Android), enterprise, and cloud applications due to its portability, scalability, and rich ecosystem.

4. References:

1. **Oracle Official Java Documentation**
   - https://docs.oracle.com/javase/tutorial/
2. **"Java: The Complete Reference" by Herbert Schildt**
   - McGraw-Hill Education, Latest Edition
3. **GeeksforGeeks – Java Programming Language**
   - https://www.geeksforgeeks.org/java/
4. **TutorialsPoint – Java Programming**
   - https://www.tutorialspoint.com/java/

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/